Laravel Controllers, Controller Routing, Resource Controllers & Middleware

Complete Teaching Notes
Laravel 12
Beginner to Advanced

These teaching notes cover Laravel controllers from the basics to practical implementation. They are structured for classroom teaching, with definitions, syntax, step-by-step explanations, examples, complete code, and practice exercises.

Module learning objectives

After completing this topic, students will be able to:

1. Understand MVC architecture and the role of controllers.
2. Create and use controllers in Laravel.
3. Define controller-based routes.
4. Understand resource controllers and CRUD operations.
5. Apply middleware to controllers and individual methods.
6. Connect routes, controllers, and Blade views.
7. Pass data from controllers to views.
8. Build a simple CRUD application using Laravel.

1. Introduction to Controllers

1.1 What is a Controller?

A Controller is a PHP class in Laravel that handles incoming HTTP requests, processes application logic, and returns an appropriate response.

In simple words:

- Route receives the URL request.
- Controller handles the request.
- Model communicates with the database (when needed).
- View displays the output to the user.

Example:

Suppose a user visits:

```
http://127.0.0.1:8000/students
```

The request flow is:
User / Browser
Requests /students
Route (web.php)
Matches the URL and HTTP method
Controller (StudentController)
Processes the request
Model (Student)
Fetches data from the database, if required
View (students.blade.php)
Displays the result
Browser displays the response
Note: The Model is optional. A controller can return a view directly without accessing a database.

1.2 Why do we need Controllers?

Without controllers, we might write all application logic directly inside route definitions.

Without Controller:

```
use Illuminate\Support\Facades\Route;

Route::get('/welcome', function () {
    $name = "Mehebub";

    return view('welcome', compact('name'));
});
```

This works, but as an application grows, keeping all logic inside routes makes the code difficult to maintain.
With Controller:

```
Route::get('/welcome', [WelcomeController::class, 'index']);
```

Controller:

```
namespace App\Http\Controllers;

class WelcomeController extends Controller
{
    public function index()
    {
        $name = "Mehebub";

        return view('welcome', compact('name'));
    }
}
```

Advantages of Controllers:

- Clean and organized code.
- Reusable application logic.
- Easier debugging and maintenance.
- Separation of responsibilities.
- Better integration with middleware and models.

2. Creating Controllers

Laravel provides an Artisan command to generate controllers automatically.

2.1 Create a Basic Controller

Open the terminal inside your Laravel project.
Command:

```
php artisan make:controller StudentController
```

After executing the command, Laravel creates:

```
app/
 └── Http/
      └── Controllers/
           ├── Controller.php
           └── StudentController.php
```

Generated controller:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class StudentController extends Controller
{
    //
}
```

Understanding the code

Code	Explanation
<?php	Starts PHP code.
namespace	Defines the controller's namespace.
use Illuminate\Http\Request	Imports the Request class.
class StudentController	Declares the controller class.
extends Controller	Inherits from Laravel's base controller.

2.2 Adding Methods to a Controller

A method is a function defined inside a class.
Example:

```
<?php

namespace App\Http\Controllers;

class StudentController extends Controller
{
    public function index()
    {
        return ”Student List”;
    }

    public function create()
    {
        return ”Create Student”;
    }

    public function show()
    {
        return ”Student Details”;
    }
}
```

Here, we have created three methods:

- index() – displays the student list.
- create() – displays the student creation page.
- show() – displays student details.

2.3 Connecting Controller with Routes

Open:

```
routes/web.php
```

Add:

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\StudentController;

Route::get('/students', [StudentController::class, 'index']);

Route::get('/students/create', [StudentController::class, 'create']);

Route::get('/students/show', [StudentController::class, 'show']);
```

Syntax

```
Route::get('/URL', [ControllerName::class, 'methodName']);
```

Explanation:

- Route::get() – defines a GET route.
- '/students' – URL accessed by the browser.
- StudentController::class – specifies the controller class.
- 'index' – specifies the method to execute.

2.4 Testing the Routes

Start the Laravel development server:

```
php artisan serve
```

Open the following URLs:

URL	Expected output
/students	Student List
/students/create	Create Student
/students/show	Student Details

To see all registered routes:

```
php artisan route:list
```

3. Controller Routing

Controller routing means connecting application URLs to specific controller methods. Instead of writing application logic inside routes, we define the route and assign the task to a controller.

3.1 GET Route

GET routes are generally used to retrieve information or display pages.

```
use App\Http\Controllers\StudentController;

Route::get('/students', [StudentController::class, 'index']);
```

Controller:

```
public function index()
{
    return "Display all students";
}
```

Browser:

```
http://127.0.0.1:8000/students
```

Output:

```
Display all students
```

3.2 POST Route

POST routes are generally used to submit data to the server.
Route:

```
Route::post('/students', [StudentController::class, 'store']);
```

Controller:

```
public function store(Request $request)
{
    return "Student data submitted";
}
```

Example HTML form:

```
<form action="/students" method="POST">
    <input type="text" name="name" placeholder="Student Name">
    <button type="submit">Save</button>
</form>
```

Because Laravel enables CSRF protection for web routes, include the CSRF token in Blade forms:

```
<form action="/students" method="POST">
    @csrf
    <input type="text" name="name">
    <button type="submit">Save</button>
</form>
```

Without a valid CSRF token, Laravel normally returns HTTP 419 for a POST request through the web middleware group.

3.3 Route Parameters with Controllers

Route parameters allow us to pass dynamic values through URLs.
Example:

```
/students/101
/students/102
/students/103
```

Route:

```
Route::get('/students/{id}', [StudentController::class, 'show']);
```

Controller:

```
public function show($id)
{
    return "Student ID: " . $id;
}
```

Output for /students/101:

```
Student ID: 101
```

Multiple Route Parameters

Route:

```
Route::get(
    '/students/{id}/{name}',
    [StudentController::class, 'details']
);
```

Controller:

```
public function details($id, $name)
{
    return "ID: $id, Name: $name";
}
```

URL:

```
/students/101/Mehebub
```

Output:

```
ID: 101, Name: Mehebub
```

Important: The controller method parameters must follow the order of the route parameters.

3.4 Named Routes with Controllers

Named routes allow us to give a unique name to a route and generate its URL without hardcoding the URL.
Route:

```
Route::get('/students', [StudentController::class, 'index'])
    ->name('students.index');
```

Generate URL in Blade:

```
<a href="{{ route('students.index') }}">
    View Students
</a>
```

Redirect using a named route:

```
return redirect()->route('students.index');
```

Named routes are useful because if the URL changes later, we generally don't need to update every link that uses the route name.

4. Passing Data from Controller to View

A controller can pass variables and data to a Blade view.

4.1 Create a View

Create this file:

```
resources/views/students.blade.php
```

Add:

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Page</title>
</head>
<body>

    <h1>Welcome to Student Page</h1>

</body>
</html>
```

4.2 Return View from Controller

Controller:

```
public function index()
{
    return view('students');
}
```

Route:

```
Route::get('/students', [StudentController::class, 'index']);
```

Explanation:
view['students'] tells Laravel to load:

```
resources/views/students.blade.php
```

You do not need to write the .blade.php extension.

4.3 Passing a Single Variable

Controller:

```
public function index()
{
    $name = "Mehebub Alam";

    return view('students', compact('name'));
}
```

View:

```
<h1>Welcome, {{ $name }}</h1>
```

Output:

```
Welcome, Mehebub Alam
```

Alternative syntax:

```
return view('students', ['name' => $name]);
```

Both methods pass data to the view.

4.4 Passing Multiple Variables

Controller:

```
public function index()
{
    $name = "Mehebub";
    $course = "Laravel";
    $duration = "3 Months";

    return view('students', compact(
        'name',
        'course',
        'duration'
    ));
}
```

View:

```
<h1>Name: {{ $name }}</h1>

<h2>Course: {{ $course }}</h2>

<h3>Duration: {{ $duration }}</h3>
```

4.5 Passing an Array to View

Controller:

```
public function index()
{
    $students = [
        "Mehebub",
        "Rahul",
        "Akash",
        "Rohan",
        "Suman"
    ];

    return view('students', compact('students'));
}
```

View:

```
<h2>Student List</h2>

@foreach($students as $student)

    <p>{{ $student }}</p>

@endforeach
```

Output:

```
Student List

Mehebub
Rahul
Akash
Rohan
Suman
```

4.6 Passing Associative Arrays

Controller:

```
public function index()
{
    $student = [
        'name' => 'Mehebub Alam',
        'email' => 'mehebub@gmail.com',
        'course' => 'Laravel Development',
        'age' => 23
    ];

    return view('students', compact('student'));
}
```

View:

```
<h2>Student Details</h2>

<p>Name: {{ $student['name'] }}</p>

<p>Email: {{ $student['email'] }}</p>

<p>Course: {{ $student['course'] }}</p>

<p>Age: {{ $student['age'] }}</p>
```

5. Resource Controllers

5.1 What is a Resource Controller?

A Resource Controller is a special Laravel controller that provides predefined methods for performing CRUD operations.
CRUD stands for:

Operation	Meaning	HTTP method
Create	Insert new data	POST
Read	Retrieve data	GET
Update	Modify existing data	PUT / PATCH
Delete	Remove data	DELETE

Instead of manually creating separate routes for every operation, Laravel allows us to generate a resource controller and register all its routes using one command.

5.2 Create a Resource Controller

Run:

```
php artisan make:controller StudentController --resource
```

Laravel generates a controller containing seven predefined methods.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class StudentController extends Controller
{
    public function index()
    {
        //
    }

    public function create()
    {
        //
    }

    public function store(Request $request)
    {
        //
    }

    public function show(string $id)
    {
        //
    }

    public function edit(string $id)
    {
        //
    }

    public function update(Request $request, string $id)
    {
        //
    }

    public function destroy(string $id)
    {
        //
    }
}
```

5.3 Understanding the Seven Resource Methods

1. index()
GET
/students
Displays all students.

2. create()
GET
/students/create
Displays the form to create a student.

3. store()
POST
/students
Receives and saves new student information.

4. show()
GET
/students/{student}
Displays details of a particular student.

5. edit()
GET
/students/{student}/edit

Displays the form to edit an existing student.

6. update()

PUT / PATCH

/students/{student}

Updates an existing student's information.

7. destroy()

DELETE

/students/{student}

Deletes a particular student.

Explanation with examples

1. index() – Display all students

```
public function index()
{
    return ”Display all students”;
}
```

2. create() – Display the student form

```
public function create()
{
    return view(’students.create’);
}
```

3. store() – Save a new student

```
public function store(Request $request)
{
    $name = $request->name;
    return ”Student saved: ” . $name;
}
```

4. show() – Display a particular student

```
public function show(string $id)
{
    return ”Student ID: ” . $id;
}
```

5. edit() – Display the edit form

```
public function edit(string $id)
{
    return ”Edit Student ID: ” . $id;
}
```

6. update() – Update student information

```
public function update(Request $request, string $id)
{
    return ”Updated Student ID: ” . $id;
}
```

7. destroy() – Delete a student

```
public function destroy(string $id)
{
    return ”Deleted Student ID: ” . $id;
}
```

These are demonstration responses. Actual CRUD methods will be implemented later using database operations.

5.4 Register Resource Routes

Open routes/web.php.

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\StudentController;
Route::resource(’students’, StudentController::class);
```

That's it! Laravel automatically generates all seven routes.
Check the routes:

```
php artisan route:list
```

Expected resource routes:

Method	URL	Controller method
GET	/students	index
GET	/students/create	create
POST	/students	store
GET	/students/{student}	show
GET	/students/{student}/edit	edit
PUT/ PATCH	/students/{student}	update
DELETE	/students/{student}	destroy

5.5 Resource Controller Route Naming

Laravel also automatically assigns route names.

Route name	Purpose
students.index	Student list
students.create	Create form
students.store	Save student
students.show	Student details
students.edit	Edit form
students.update	Update student
students.destroy	Delete student

Example:

```
<a href=”{{ route(’students.index’) }}”>
    All Students
</a>
<a href=”{{ route(’students.create’) }}”>
    Add Student
</a>
```

5.6 Resource Controller with Only and Except

Sometimes we don't need all seven methods.
Using only[]:

```
Route::resource('students', StudentController::class)
    ->only(['index', 'show']);
```

Only two routes will be registered.
Using except[]:

```
Route::resource('students', StudentController::class)
    ->except(['destroy']);
```

All resource routes will be registered except the delete route.

5.7 Resource Controller with a Custom Name

```
Route::resource('students', StudentController::class)
    ->names([
        'index' => 'student.list',
        'create' => 'student.add'
    ]);
```

Now the named routes will include:

```
route('student.list');

route('student.add');
```

6. Controller Middleware

6.1 What is Middleware?

Middleware is a mechanism that filters or checks incoming HTTP requests before they reach the application's intended destination.
In simple words, middleware works like a security checkpoint.
Real-life example:
Imagine entering a college:

1. A student arrives at the entrance.
2. The security guard checks the student's ID card.
3. If the ID is valid, the student is allowed inside.
4. Otherwise, entry is denied.

Similarly, Laravel middleware checks requests before allowing them to proceed.

6.2 Middleware Request Flow

User Request
Middleware
Checks authentication, permissions, etc.
Allowed
Continue to route/controller
Rejected
Redirect or return error

6.3 Common Uses of Middleware

- Authentication: Check whether the user is logged in.
- Authorization: Check whether the user has permission.
- Admin access: Allow only administrators.
- Role checking: Allow specific user roles.
- Request filtering: Reject unwanted requests.
- Rate limiting: Restrict excessive requests.

Laravel provides several built-in middleware, including:

Middleware	Purpose
auth	Checks whether the user is authenticated.
guest	Allows only unauthenticated users.
verified	Checks email verification.
throttle	Limits request frequency.
can	Checks authorization permissions.

6.4 Creating Custom Middleware

Use Artisan:

```
php artisan make:middleware CheckAge
```

Laravel creates:

```
app/Http/Middleware/CheckAge.php
```

Generated structure:

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class CheckAge
{
    public function handle(Request $request, Closure $next): Response
    {
        return $next($request);
    }
}
```

Understanding handle()

```
public function handle(Request $request, Closure $next): Response
```

- Request $request: Contains the incoming HTTP request.
- Closure $next: Passes the request to the next middleware or destination.
- Response: Represents the HTTP response.

Example: Check User Age

Modify CheckAge.php:

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class CheckAge
{
    public function handle(Request $request, Closure $next): Response
    {
        if ($request->age < 18) {
            return response('Access Denied: You must be 18 or older.', 403);
        }

        return $next($request);
    }
}
```

Explanation:

- If age is less than 18, access is denied.
- If age is 18 or above, the request continues.

This example is for teaching. In a real application, don't trust an age supplied directly through a URL or form for security decisions.

6.5 Registering Middleware in Laravel 12

In Laravel 12, custom middleware aliases are registered in:

```
bootstrap/app.php
```

Find the withMiddleware[] section and register the alias:

```
use App\Http\Middleware\CheckAge;
use Illuminate\Foundation\Configuration\Middleware;
```

Inside the application configuration:

```
->withMiddleware(function (Middleware $middleware): void {
    $middleware->alias([
        'checkage' => CheckAge::class,
    ]);
})
```

The complete relevant structure:

```
<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;
use App\Http\Middleware\CheckAge;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->alias([
            'checkage' => CheckAge::class,
        ]);
    })
```

```
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
```

Important: If your project already contains middleware configuration, add the alias inside the existing withMiddleware() closure. Do not create a second configuration block unnecessarily.

6.6 Applying Middleware to Routes

Once registered, use the middleware alias in web.php.

```
Route::get('/students', [StudentController::class, 'index'])
    ->middleware('checkage');
```

You can also apply middleware to a group:

```
Route::middleware(['checkage'])->group(function () {

    Route::get('/students', [StudentController::class, 'index']);

    Route::get('/students/create', [StudentController::class, 'create']);

});
```

Every route inside the group uses the middleware.

6.7 Applying Middleware to a Controller

Laravel 12 supports controller middleware through the HasMiddleware interface. Example:

```
<?php

namespace App\Http\Controllers;

use App\Http\Middleware\CheckAge;
use Illuminate\Routing\Controllers\HasMiddleware;
use Illuminate\Routing\Controllers\Middleware;

class StudentController extends Controller implements HasMiddleware
{
    public static function middleware(): array
    {
        return [
            new Middleware('checkage'),
        ];
    }

    public function index()
    {
        return "Student List";
    }

    public function create()
    {
        return "Create Student";
    }
}
```

Now the middleware applies to the controller's actions.

Apply Middleware to Specific Controller Methods

Suppose we want to apply middleware only to create() and store().

```
public static function middleware(): array
{
    return [
        new Middleware('checkage', only: ['create', 'store']),
    ];
}
```

Apply Middleware Except Specific Methods

```
public static function middleware(): array
{
    return [
        new Middleware('checkage', except: ['index']),
    ];
}
```

Here, all controller methods except index() use the middleware.
Version note
Older Laravel tutorials often show $this->middleware('auth') inside the controller constructor. That pattern is not the standard approach for fresh Laravel 11/12 applications using the default base controller. Use HasMiddleware or attach middleware to routes instead.

7. Connecting Routes, Controllers and Views

This is one of the most important parts of Laravel MVC development.
Let's build a small Student Management example.

7.1 Project Structure

We will create the following files:

```
app/
  └──── Http/
          └──── Controllers/
                  └──── StudentController.php
resources/
  └──── views/
          └──── students/
                  ├──── index.blade.php
                  ├──── create.blade.php
                  └──── show.blade.php
routes/
  └──── web.php
```

7.2 Step 1: Create Controller

```
php artisan make:controller StudentController
```

7.3 Step 2: Create Routes

Open routes/web.php.

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\StudentController;

Route::get('/students', [StudentController::class, 'index'])
    ->name('students.index');

Route::get('/students/create', [StudentController::class, 'create'])
    ->name('students.create');

Route::get('/students/{id}', [StudentController::class, 'show'])
    ->name('students.show');
```

7.4 Step 3: Create the Controller Logic

Open:

```
app/Http/Controllers/StudentController.php
```

Add:

```
<?php

namespace App\Http\Controllers;

class StudentController extends Controller
{
    public function index()
    {
        $students = [
            [
                'id' => 1,
                'name' => 'Rahul',
                'course' => 'PHP'
            ],
            [
                'id' => 2,
                'name' => 'Mehebub',
                'course' => 'Laravel'
            ],
            [
                'id' => 3,
                'name' => 'Akash',
                'course' => 'React'
            ]
        ];

        return view('students.index', compact('students'));
    }

    public function create()
    {
        return view('students.create');
    }

    public function show($id)
    {
        $student = [
            'id' => $id,
            'name' => 'Mehebub',
            'course' => 'Laravel'
        ];

        return view('students.show', compact('student'));
    }
}
```

7.5 Step 4: Create the Student List View

Create:

```
resources/views/students/index.blade.php
```

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Student List</title>
</head>
<body>

    <h1>Student Management System</h1>

    <a href="{{ route('students.create') }}">
        Add New Student
    </a>

    <hr>

    <h2>All Students</h2>

    <table border="1" cellpadding="10">

        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Course</th>
            <th>Action</th>
        </tr>

        @foreach($students as $student)

        <tr>
            <td>{{ $student['id'] }}</td>

            <td>{{ $student['name'] }}</td>

            <td>{{ $student['course'] }}</td>

            <td>
                <a href="{{ route('students.show', $student['id']) }}">
                    View
                </a>
            </td>
        </tr>

        @endforeach

    </table>

</body>
</html>
```

7.6 Step 5: Create the Student Form View

Create:

```
resources/views/students/create.blade.php
```

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Create Student</title>
</head>
<body>
    <h1>Add New Student</h1>
    <form action="#" method="POST">
        @csrf
        <label>Student Name:</label>
        <input type="text" name="name">
        <br><br>
        <label>Course:</label>
        <input type="text" name="course">
        <br><br>
        <button type="submit">Save Student</button>
    </form>
    <br>
    <a href="{{ route('students.index') }}">
        Back to Student List
    </a>
</body>
</html>
```

Note: This is only the form interface. It does not save data because we have not connected a POST route or database yet.

7.7 Step 6: Create Student Details View

Create:

```
resources/views/students/show.blade.php
```

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Details</title>
</head>
<body>
    <h1>Student Details</h1>
    <p>
        ID: {{ $student['id'] }}
    </p>
    <p>
        Name: {{ $student['name'] }}
    </p>
    <p>
```

```
        Course: {{ $student['course'] }}
    </p>

    <a href="{{ route('students.index') }}”>
        Back to Students
    </a>

</body>
</html>
```

7.8 Step 7: Run the Application

```
php artisan serve
```

Visit:

```
http://127.0.0.1:8000/students
```

Expected application structure:
Student Management System Frontend with · Built with Blink
blink.new
The application will display a list of students with a View option and an Add New Student link.
Important teaching point: This example uses PHP arrays, not a database. It demonstrates how data moves from routes to controllers and then to views. The data will reset when the request ends.

8. Practical Project: Complete Student CRUD Using Laravel

Now we will build an actual CRUD application using a database, an Eloquent Model, a Resource Controller, Routes, and Blade Views.
This is the most important practical example for understanding how all MVC components work together.

8.1 Application workflow

Blade View
Student forms and student list
Resource Routes
GET, POST, PUT/PATCH, DELETE
StudentController
Handles CRUD operations and validation
Student Model
Communicates with the database
MySQL Database
students table

8.2 Step 1: Create Model, Migration and Controller

Run:

```
php artisan make:model Student -mcr
```

Explanation:

Option	Meaning
make:model	Creates a Model.
Student	Model name.
-m	Creates a migration.
-c	Creates a controller.
-r	Creates a resource controller.

Generated files:

```
app/
  ├──── Models/
  │    └──── Student.php
  │
  └──── Http/
        └──── Controllers/
              └──── StudentController.php
database/
  └──── migrations/
        └──── create_students_table.php
```

8.3 Step 2: Configure MySQL Database

Open .env:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel_school
DB_USERNAME=root
DB_PASSWORD=
```

Create a MySQL database named:

```
CREATE DATABASE laravel_school;
```

Make sure your MySQL server is running.

8.4 Step 3: Create the Students Table

Open the generated migration file inside:

```
database/migrations/
```

Add:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('students', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
```

```
            $table->string('course');

            $table->timestamps();
        });
    }
    public function down(): void
    {
        Schema::dropIfExists('students');
    }
};
```

Understanding the migration

Code	Explanation
Schema::create()	Creates a database table.
$table->id()	Creates an auto-incrementing primary key.
$table->string('name')	Creates a string column.
$table->string('email')->unique()	Creates a unique email column.
$table->string('course')	Creates a course column.
$table->timestamps()	Creates created_at and updated_at.
down()	Defines how to reverse the migration.

Run:

```
php artisan migrate
```

Now the students table will be created.

8.5 Step 4: Configure the Student Model

Open:

```
app/Models/Student.php
```

Add:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    protected $fillable = [
        'name',
        'email',
        'course'
    ];
}
```

What is $fillable?

$fillable specifies which model attributes can be assigned through mass assignment. For example:

```
Student::create([
    ’name’ => ’Mehebub’,
    ’email’ => ’mehebub@gmail.com’,
    ’course’ => ’Laravel’
]);
```

Only the attributes permitted by $fillable can be mass assigned.

8.6 Step 5: Register Resource Routes

Open routes/web.php:

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\StudentController;

Route::resource(’students’, StudentController::class);
```

Laravel automatically creates the seven CRUD routes.

8.7 Step 6: Write the Controller Logic

Open:

```
app/Http/Controllers/StudentController.php
```

Replace its contents with:

```
<?php

namespace App\Http\Controllers;

use App\Models\Student;
use Illuminate\Http\Request;

class StudentController extends Controller
{
    // Display all students
    public function index()
    {
        $students = Student::latest()->get();

        return view(’students.index’, compact(’students’));
    }

    // Display student creation form
    public function create()
    {
        return view(’students.create’);
    }

    // Store a new student
    public function store(Request $request)
    {
        $validated = $request->validate([
            ’name’ => ’required|string|max:255’,
            ’email’ => ’required|email|unique:students,email’,
            ’course’ => ’required|string|max:255’
        ]);

        Student::create($validated);

        return redirect()
            ->route(’students.index’)
            ->with(’success’, ’Student added successfully!’);
    }

    // Display a particular student
    public function show(Student $student)
    {
        return view(’students.show’, compact(’student’));
```

```
    }

    // Display edit form
    public function edit(Student $student)
    {
        return view('students.edit', compact('student'));
    }

    // Update student information
    public function update(Request $request, Student $student)
    {
        $validated = $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:students,email,' . $student->id,
            'course' => 'required|string|max:255'
        ]);

        $student->update($validated);

        return redirect()
            ->route('students.index')
            ->with('success', 'Student updated successfully!');
    }

    // Delete a student
    public function destroy(Student $student)
    {
        $student->delete();

        return redirect()
            ->route('students.index')
            ->with('success', 'Student deleted successfully!');
    }
}
```

Important controller concepts

A. Retrieve all records

```
$students = Student::all();
```

Retrieves all student records.

Another option:

```
$students = Student::latest()->get();
```

Retrieves students ordered by latest creation date.

B. Insert a record

```
Student::create($validated);
```

Inserts a new student record.

C. Find a particular record

```
Student::find(1);
```

Finds the student whose primary key is 1. Returns null if not found.

D. Update a record

```
$student->update($validated);
```

Updates the selected student.

E. Delete a record

```
$student->delete();
```

Deletes the selected student.

F. Validate form data

```
$request->validate([
    'name' => 'required|string|max:255',
    'email' => 'required|email'
]);
```

Validates incoming form data before processing it.

Understanding Route Model Binding

Notice these methods:

```
public function show(Student $student)
```

Instead of writing:

```
public function show($id)
{
    $student = Student::findOrFail($id);
}
```

Laravel can automatically find the corresponding Student model using implicit route model binding.

For example:

```
/students/5
```

Laravel finds the student with ID 5 and injects the model into the controller method.

If that record does not exist, Laravel returns a 404 response.

8.8 Step 7: Create Blade Views

Create a folder:

```
resources/views/students/
```

We need four files:

```
students/
  ├── index.blade.php
  ├── create.blade.php
  ├── edit.blade.php
  └── show.blade.php
```

A. index.blade.php – Display Students

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Management</title>
</head>
<body>

    <h1>Student Management System</h1>

    @if(session(’success’))
        <p style=”color: green”>
            {{ session(’success’) }}
        </p>
    @endif

    <a href=”{{ route(’students.create’) }}”>
        Add Student
    </a>

    <hr>

    <table border=”1” cellpadding=”10”>

        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Course</th>
            <th>Action</th>
        </tr>

        @foreach($students as $student)

        <tr>
            <td>{{ $student->id }}</td>

            <td>{{ $student->name }}</td>

            <td>{{ $student->email }}</td>

            <td>{{ $student->course }}</td>

            <td>
                <a href=”{{ route(’students.show’, $student->id) }}”>
                    View
                </a>

                <a href=”{{ route(’students.edit’, $student->id) }}”>
                    Edit
                </a>

                <form action=”{{ route(’students.destroy’, $student->id) }}”
                      method=”POST”
                      style=”display:inline”>

                    @csrf

                    @method(’DELETE’)

                    <button type=”submit”>Delete</button>

                </form>
            </td>
        </tr>

        @endforeach

    </table>

</body>
</html>
```

B. create.blade.php – Add Student

```
<!DOCTYPE html>
<html>
<head>
    <title>Add Student</title>
</head>
<body>

    <h1>Add New Student</h1>

    <form action=”{{ route(’students.store’) }}” method=”POST”>

        @csrf

        <label>Name:</label>

        <input type=”text” name=”name” value=”{{ old(’name’) }}”>

        @error(’name’)
            <p style=”color:red”>{{ $message }}</p>
        @enderror

        <br><br>

        <label>Email:</label>

        <input type=”email” name=”email” value=”{{ old(’email’) }}”>

        @error(’email’)
            <p style=”color:red”>{{ $message }}</p>
        @enderror

        <br><br>

        <label>Course:</label>

        <input type=”text” name=”course” value=”{{ old(’course’) }}”>

        @error(’course’)
            <p style=”color:red”>{{ $message }}</p>
        @enderror

        <br><br>

        <button type=”submit”>Save Student</button>

    </form>

</body>
</html>
```

C. edit.blade.php – Update Student

```
<!DOCTYPE html>
<html>
<head>
    <title>Edit Student</title>
</head>
<body>

    <h1>Edit Student</h1>

    <form action="{{ route('students.update', $student->id) }}"
          method="POST">

        @csrf

        @method('PUT')

        <label>Name:</label>

        <input type="text" name="name"
               value="{{ old('name', $student->name) }}">

        @error('name')
            <p style="color:red">{{ $message }}</p>
        @enderror

        <br><br>

        <label>Email:</label>

        <input type="email" name="email"
               value="{{ old('email', $student->email) }}">

        @error('email')
            <p style="color:red">{{ $message }}</p>
        @enderror

        <br><br>

        <label>Course:</label>

        <input type="text" name="course"
               value="{{ old('course', $student->course) }}">

        @error('course')
            <p style="color:red">{{ $message }}</p>
        @enderror

        <br><br>

        <button type="submit">Update Student</button>

    </form>

</body>
</html>
```

D. show.blade.php – Student Details

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Details</title>
</head>
<body>

    <h1>Student Details</h1>

    <p>ID: {{ $student->id }}</p>

    <p>Name: {{ $student->name }}</p>

    <p>Email: {{ $student->email }}</p>

    <p>Course: {{ $student->course }}</p>

    <a href="{{ route('students.index') }}">
        Back to Students
    </a>

</body>
</html>
```

8.9 Step 8: Run and Test the Application

Run:

```
php artisan serve
```

Open:

```
http://127.0.0.1:8000/students
```

GitHub - RezalSultan/Website-Sistem-Pengelolaan-Data-Mahasiswa
github.com
Test these operations:

1. Click Add Student and insert a student.
2. Check whether the student appears in the list.
3. Click View to see student details.
4. Click Edit and modify the student's information.
5. Click Delete to remove the student.
6. Try submitting an empty form and observe validation errors.

Practical outcome
You have now connected all five important Laravel components:

- Routes
- Controllers
- Models
- Views
- Database

9. Important Laravel Concepts Used in Controllers

9.1 Request Class

The Request class provides access to incoming HTTP request information.
Import:

```
use Illuminate\Http\Request;
```

Example:

```
public function store(Request $request)
{
    $name = $request->input('name');
    $email = $request->input('email');
    return "Name: $name, Email: $email";
}
```

Alternative syntax:

```
$name = $request->name;
```

Useful Request methods:

Method	Purpose
$request->all[]	Gets all input data.
$request->input['name']	Gets a particular input.
$request->only[['name', 'email']]	Gets selected input fields.
$request->has['name']	Checks whether an input exists.
$request->filled['name']	Checks whether an input is present and not empty.
$request->method[]	Gets the HTTP request method.
$request->ip[]	Gets the request IP address.

9.2 Redirect Response

A controller can redirect the user to another route.

```
return redirect('/students');
```

Using a named route:

```
return redirect()->route('students.index');
```

9.3 Flash Messages

Flash messages are temporary session messages generally used to display success or error notifications after an operation.
Controller:

```
return redirect()
    ->route('students.index')
    ->with('success', 'Student saved successfully!');
```

Blade:

```
@if(session('success'))
    <div>
        {{ session('success') }}
    </div>
@endif
```

9.4 Abort Response

The abort[] helper can return an HTTP error response.

```
public function show($id)
{
    if ($id <= 0) {
        abort(404);
    }
    return ”Student ID: ” . $id;
}
```

Common HTTP status codes:

Code	Meaning
200	OK
201	Created
302	Redirect
403	Forbidden
404	Not Found
419	Page Expired / CSRF token mismatch in common Laravel web requests
422	Validation error
500	Internal Server Error

10. Difference Between Normal and Resource Controllers

Normal Controller	Resource Controller
Created using make:controller	Created using make:controller --resource
Methods are manually defined	Seven CRUD methods are generated
Routes are manually registered	All CRUD routes can be registered using Route::resource()
Suitable for custom application logic	Suitable for CRUD applications
More flexibility in route setup	Faster CRUD development

11. Important Artisan Commands – Revision

Command	Purpose
php artisan make:controller StudentController	Create a normal controller
php artisan make:controller StudentController --resource	Create a resource controller
php artisan make:middleware CheckAge	Create custom middleware
php artisan make:model Student	Create a model
php artisan make:model Student -mcr	Create model, migration and resource controller
php artisan migrate	Execute migrations
php artisan route:list	Display registered routes
php artisan route:clear	Clear route cache
php artisan optimize:clear	Clear optimization caches

12. Important Interview and Viva Questions

Q1. What is a Controller in Laravel?
A controller is a PHP class responsible for handling application requests and processing business logic.
Q2. Which Artisan command creates a controller?

```
php artisan make:controller StudentController
```

Q3. What is the difference between a route and a controller?
A route determines which application action should handle a URL, while a controller executes the associated application logic.
Q4. What is a Resource Controller?
A resource controller provides seven predefined methods for implementing CRUD operations.
Q5. What are the seven resource controller methods?
index(), create(), store(), show(), edit(), update(), and destroy().
Q6. How do you register resource routes?

```
Route::resource('students', StudentController::class);
```

Q7. What is Middleware?
Middleware filters incoming HTTP requests before they reach their intended destination.

Q8. Where do we register custom middleware aliases in Laravel 12?
In bootstrap/app.php, inside the withMiddleware() configuration.
Q9. How do you pass data from a controller to a view?

```
return view('students', compact('name'));
```

Q10. What is the purpose of compact()?
It creates an associative array from existing PHP variables, using their variable names as array keys.
Q11. What is the purpose of $request?
It provides access to incoming HTTP request data.
Q12. What is Route Model Binding?
It automatically resolves a route parameter into the corresponding Eloquent model.
Q13. What is the purpose of @csrf?
It inserts a CSRF token into a form to protect against Cross-Site Request Forgery.
Q14. What is the difference between PUT and PATCH?
PUT generally represents replacing an entire resource, while PATCH represents partially updating a resource.

Q15. Can we apply middleware to specific controller methods?
Yes. Laravel 12 supports the HasMiddleware interface with only and except options.

13. Classroom Practical Exercises

Student Practice
These exercises can be assigned after completing the lesson.

Exercise 1 – Basic Controller

1. Create a controller named TeacherController.
2. Create three methods: index(), create(), and show().
3. Return different messages from each method.
4. Connect all three methods with GET routes.
5. Test the routes in a browser.

Exercise 2 – Controller and View

1. Create a CourseController.
2. Create a view named courses.blade.php.
3. Pass course name, duration, fees and instructor name from the controller.
4. Display all information using Blade syntax.

Exercise 3 – Resource Controller

1. Create a BookController using the resource option.
2. Register resource routes.
3. Display all seven routes using route:list.
4. Explain the purpose of each method.

Exercise 4 – Custom Middleware

1. Create middleware named CheckAge.
2. Allow access only when the age is 18 or above.
3. Register its alias.
4. Apply it to the /admission route.
5. Test both allowed and rejected requests.

Exercise 5 – Mini Project

Build a Student Management System.
Requirements:

- Add student.
- View all students.
- View individual student details.
- Edit student information.
- Delete student information.
- Validate form fields.
- Display success and error messages.
- Use a Resource Controller.
- Use MySQL database.
- Use Blade Views.

14. Final Revision: Complete MVC Request Flow

Laravel MVC Architecture

1. User sends a request
1. Example: GET /students
2. Middleware checks the request
2. Authentication, authorization, and other checks.
3. Route identifies the controller
3. StudentController@index
4. Controller processes the request
4. Executes application logic.
5. Model communicates with database
5. Fetches or modifies student records, if required.
6. Controller returns a response
6. Usually a Blade view with data.
7. Browser displays the result
7. User sees the student list.

Key takeaways for students

1. Route: Determines where the request goes.
2. Controller: Handles the application logic.
3. Model: Handles database interaction.
4. View: Displays information to the user.
5. Middleware: Filters and checks incoming requests.
6. Resource Controller: Provides seven standard CRUD methods.
7. Blade: Laravel's templating engine for creating dynamic views.